Started on	Tuesday, 17 January 2023, 9:57 PM
State	Finished
Completed on	Tuesday, 17 January 2023, 10:01 PM
Time taken	3 mins 57 secs
Grade	**6.00** out of 10.00 (**60**%)

Question **1**

Incorrect

Mark 0.00 out of 2.00

Why is a signal considered an asynchronous operation?

Select one:

- a. Because there is no synchronization mechanism between the signal and the operating system. ✖
- b. Signals are not asynchronous.
- c. Because signals use asynchronous socket operations.
- d. Because we register a handler function and this is called when the signal is received.

Signals are considered asynchronous because they do not happen synchronously with the execution of the code. For a signal, a handler function is registered that will be run when a signal is received. The signal is usually transmitted by an external event.

The correct answer is: Because we register a handler function and this is called when the signal is received.

Question **2**

Correct

Mark 2.00 out of 2.00

Can a pipe be used after `execve()` has been called in the child process?

Select one:

- a. Yes, because the `execve()` system call does not close file descriptors by default. ✔
- b. No, because the `execve()` system call closes all file descriptors except STDIN, STDOUT and STDERR.
- c. No, because the `execve()` system call changes the entire Virtual Address Space.
- d. No, because the `execve()` system call closes all file descriptors.

From the `execve()` man page: "By default, file descriptors remain open across an `execve()`. File descriptors that are marked close-on-exec are closed."

The correct answer is: Yes, because the `execve()` system call does not close file descriptors by default.

Question **3**

Incorrect

Mark 0.00 out of 2.00

Can we use semaphores as an inter-thread communication mechanism?

Select one:

- a. No. Because we cannot use the `wait()` and `post()` methods in two different threads.
- b. No. Because we cannot send text messages using semaphores.
- c. Yes. Because we can use the semaphore for exclusive access to a shared resource. ✖
- d. Yes. Because we can use the `wait()` and `post()` methods in two different threads.

Inter-thread communication means communication between two or more threads. So while you can use a semaphore to get exclusive access to a shared resource, that's not its primary purpose. Semaphores are used as signalling mechanisms. The `wait()` and `post()` methods are used precisely in this sense. One thread will wait using the `wait()` method, and another will call `post()` to notify the first one. Finally, we can also conclude that communication is not limited to the transfer of text messages.

The correct answer is: Yes. Because we can use the `wait()` and `post()` methods in two different threads.

Question **4**

Correct

Mark 2.00 out of 2.00

What is an advantage of using processes instead of threads when we want to modularize an application?

Select one:

- a. There is no advantage of using processes over threads.
- b. Processes can run on multiple CPUs, but threads cannot.
- c. Processes are a better way of confinement due to individual Virtual Address Spaces. ✔
- d. Processes can share memory more easily than threads using the `mmap()` syscall with the `MAP_SHARED` flag.

Both processes and threads can run on several CPUs (n.b. as long as they are kernel-level threads). Both processes and threads are created using the `clone()` syscall (e.g. `pthread_create()` will eventually call `clone()`). When creating a thread, we specify some flags to the `clone()` syscall so that the new thread will use the same VAS. So the shared memory mechanism for threads is ensured by the Kernel. We don't need to do anything more. The correct answer is, therefore, that the processes offer better confinement due to the use of a Virtual Address Space for each process.

The correct answer is: Processes are a better way of confinement due to individual Virtual Address Spaces.

Question **5**

Correct

Mark 2.00 out of 2.00

Why is modularity an important concept in designing an application?

Select one:

- a. Modular code is reusable.
- b. Modular code is easier to debug.
- c. All apply. ✔
- d. A modular application is easier to maintain.

Modularity is the characteristic of an application to be split into several components. This makes the code easier to maintain and easier to debug. At the same time, different parts of the application can be reused.

The correct answer is: All apply.

◄ Curs 11: App Interaction (part 1): Recap

Jump to...

Curs 12: App Interaction (part 2): Recap ►

Started on	Tuesday, 17 January 2023, 9:53 PM
State	Finished
Completed on	Tuesday, 17 January 2023, 9:57 PM
Time taken	4 mins 3 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 2.00 out of 2.00

Can we use semaphores as an inter-process communication mechanism?

Select one:

- a. Yes. By passing the virtual address of the semaphore through a pipe when creating a new process.
- b. Yes. By using a named semaphore. ✔
- c. Yes. But only for related processes.
- d. No. Because semaphores can be used only for threads.

Both threads and processes can use semaphores. To use a semaphore between 2 processes, a named semaphore or a shared memory region in which to put the semaphore is needed. Simply passing the virtual address from one process to another makes no sense because of the different VASs. Unrelated processes can also use named semaphores because they are associated with a path in the file system (e.g. "/path/to/shared/sem").

The correct answer is: Yes. By using a named semaphore.

Question **2**

Correct

Mark 2.00 out of 2.00

What is specific for message passing IPC (inter-process communication) mechanisms?

Select one:

- a. They are faster than shared memory IPCs.
- b. all apply
- c. They use an userspace buffer.
- d. They use syscalls like `read()`, `write()`, `recv()`, `send()`. ✔

Message passing IPCs like sockets, pipes, FIFOs use a kernel buffer and syscalls like `recv()` and `send()` for the actual communication. Since they use additional syscalls they are slower than shared memory IPCs.

The correct answer is: They use syscalls like `read()`, `write()`, `recv()`, `send()`.

Question **3**

Correct

Mark 2.00 out of 2.00

Can we use shared memory between two unrelated processes?

Select one:

- a. Yes. By using anonymous mappings.
- b. Yes. By using sockets.
- c. Yes. By using file mappings. ✔
- d. No.

We can share memory between 2 unrelated processes by using file mappings (i.e., `mmap()` with a file descriptor obtained by opening a file). Using a file mapping has a negative impact on the performance of shared memory because data must be flushed to disk. However, there are better techniques for shared memory using some sort of RAM filesystem (i.e. a filesystem that is backed by RAM, not disk). See `shm_open()`.

The correct answer is: Yes. By using file mappings.

Question **4**

Correct

Mark 2.00 out of 2.00

What is the most efficient method of inter-process communication?

Select one:

- a. Signals.
- b. Reading/Writing to files.
- c. Shared Memory. ✔
- d. Message Passing.

The inter-process communication mechanisms based on shared memory are the most efficient because once we get hold of a portion of shared memory, the effective communication is just working with memory. Unlike shared memory, message passing usually involves `send()` and `recv()` system calls. Signals also involve system calls, and writing and reading from a file involves working with IO, which is slow.

The correct answer is: Shared Memory.

Question **5**

Correct

Mark 2.00 out of 2.00

For a local communication what is better to use between UNIX and BSD sockets?

Select one:

- a. UNIX sockets because they are faster than BSD sockets. ✔
- b. BSD sockets because they are faster than UNIX sockets.
- c. BSD sockets because they use the loopback interface and UNIX sockets use files.
- d. UNIX sockets because we can't use BSD sockets locally.

For local communication it is better to use UNIX sockets because they are faster than BSD. UNIX sockets use some shared buffers in kernel space, while BSD sockets use the loopback interface for local communication.

The correct answer is: UNIX sockets because they are faster than BSD sockets.

◄ Curs 11: App Interaction (part 1): Recap

Jump to...

Curs 12: App Interaction (part 2): Recap ►

Started on	Tuesday, 17 January 2023, 5:10 PM
State	Finished
Completed on	Tuesday, 17 January 2023, 5:12 PM
Time taken	2 mins 13 secs
Grade	**6.00** out of 10.00 (**60**%)

Question **1**

Incorrect

Mark 0.00 out of 2.00

For a local communication what is better to use between UNIX and BSD sockets?

Select one:

- a. UNIX sockets because they are faster than BSD sockets.
- b. BSD sockets because they use the loopback interface and UNIX sockets use files.
- c. BSD sockets because they are faster than UNIX sockets.
- d. UNIX sockets because we can't use BSD sockets locally. ✖

For local communication it is better to use UNIX sockets because they are faster than BSD. UNIX sockets use some shared buffers in kernel space, while BSD sockets use the loopback interface for local communication.

The correct answer is: UNIX sockets because they are faster than BSD sockets.

Question **2**

Correct

Mark 2.00 out of 2.00

What is the most efficient method of inter-process communication?

Select one:

- a. Signals.
- b. Reading/Writing to files.
- c. Message Passing.
- d. Shared Memory. ✔

The inter-process communication mechanisms based on shared memory are the most efficient because once we get hold of a portion of shared memory, the effective communication is just working with memory. Unlike shared memory, message passing usually involves `send()` and `recv()` system calls. Signals also involve system calls, and writing and reading from a file involves working with IO, which is slow.

The correct answer is: Shared Memory.

Question **3**

Incorrect

Mark 0.00 out of 2.00

What is the difference between a pipe and a FIFO?

Select one:

- a. There is no difference between a pipe and a FIFO ✖
- b. Pipes can only be used with related processes, while FIFOs can also be used with unrelated processes.
- c. Pipes are implemented in userspace, and FIFOs are implemented in kernel space.
- d. Pipes use file descriptors, and FIFOs use signals.

Both pipes and FIFOs use file descriptors for inter-process communication. The implementation of the two mechanisms is in the kernel, that's why we need system calls on the associated file descriptors. The fundamental difference between pipes and FIFOs is that pipes cannot be used by unrelated processes, while FIFOs (or named pipes) can.

The correct answer is: Pipes can only be used with related processes, while FIFOs can also be used with unrelated processes.

Question **4**

Correct

Mark 2.00 out of 2.00

Can we use shared memory between two unrelated processes?

Select one:

- a. Yes. By using anonymous mappings.
- b. Yes. By using file mappings. ✔
- c. No.
- d. Yes. By using sockets.

We can share memory between 2 unrelated processes by using file mappings (i.e., `mmap()` with a file descriptor obtained by opening a file). Using a file mapping has a negative impact on the performance of shared memory because data must be flushed to disk. However, there are better techniques for shared memory using some sort of RAM filesystem (i.e. a filesystem that is backed by RAM, not disk). See `shm_open()`.

The correct answer is: Yes. By using file mappings.

Question **5**

Correct

Mark 2.00 out of 2.00

What is an advantage of using processes instead of threads when we want to modularize an application?

Select one:

- a. Processes can share memory more easily than threads using the `mmap()` syscall with the `MAP_SHARED` flag.
- b. There is no advantage of using processes over threads.
- c. Processes can run on multiple CPUs, but threads cannot.
- d. Processes are a better way of confinement due to individual Virtual Address Spaces. ✔

Both processes and threads can run on several CPUs (n.b. as long as they are kernel-level threads). Both processes and threads are created using the `clone()` syscall (e.g. `pthread_create()` will eventually call `clone()`). When creating a thread, we specify some flags to the `clone()` syscall so that the new thread will use the same VAS. So the shared memory mechanism for threads is ensured by the Kernel. We don't need to do anything more. The correct answer is, therefore, that the processes offer better confinement due to the use of a Virtual Address Space for each process.

The correct answer is: Processes are a better way of confinement due to individual Virtual Address Spaces.

◄ Curs 11: App Interaction (part 1): Recap

Jump to...

Curs 12: App Interaction (part 2): Recap ►

Started on	Tuesday, 17 January 2023, 5:21 PM
State	Finished
Completed on	Tuesday, 17 January 2023, 5:26 PM
Time taken	4 mins 58 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 2.00 out of 2.00

What is the difference between a pipe and a FIFO?

Select one:

- a. Pipes can only be used with related processes, while FIFOs can also be used with unrelated processes. ✔
- b. Pipes use file descriptors, and FIFOs use signals.
- c. Pipes are implemented in userspace, and FIFOs are implemented in kernel space.
- d. There is no difference between a pipe and a FIFO

Both pipes and FIFOs use file descriptors for inter-process communication. The implementation of the two mechanisms is in the kernel, that's why we need system calls on the associated file descriptors. The fundamental difference between pipes and FIFOs is that pipes cannot be used by unrelated processes, while FIFOs (or named pipes) can.

The correct answer is: Pipes can only be used with related processes, while FIFOs can also be used with unrelated processes.

Question **2**

Correct

Mark 2.00 out of 2.00

What is the most efficient method of inter-process communication?

Select one:

- a. Message Passing.
- b. Shared Memory. ✔
- c. Signals.
- d. Reading/Writing to files.

The inter-process communication mechanisms based on shared memory are the most efficient because once we get hold of a portion of shared memory, the effective communication is just working with memory. Unlike shared memory, message passing usually involves `send()` and `recv()` system calls. Signals also involve system calls, and writing and reading from a file involves working with IO, which is slow.

The correct answer is: Shared Memory.

Question **3**

Correct

Mark 2.00 out of 2.00

For a local communication what is better to use between UNIX and BSD sockets?

Select one:

a. BSD sockets because they use the loopback interface and UNIX sockets use files.

b. BSD sockets because they are faster than UNIX sockets.

c. UNIX sockets because we can't use BSD sockets locally.

d. UNIX sockets because they are faster than BSD sockets. ✔

For local communication it is better to use UNIX sockets because they are faster than BSD. UNIX sockets use some shared buffers in kernel space, while BSD sockets use the loopback interface for local communication.

The correct answer is: UNIX sockets because they are faster than BSD sockets.

Question **4**

Correct

Mark 2.00 out of 2.00

What is an advantage of using processes instead of threads when we want to modularize an application?

Select one:

a. Processes can run on multiple CPUs, but threads cannot.

b. Processes can share memory more easily than threads using the `mmap()` syscall with the `MAP_SHARED` flag.

c. There is no advantage of using processes over threads.

d. Processes are a better way of confinement due to individual Virtual Address Spaces. ✔

Both processes and threads can run on several CPUs (n.b. as long as they are kernel-level threads). Both processes and threads are created using the `clone()` syscall (e.g. `pthread_create()` will eventually call `clone()`). When creating a thread, we specify some flags to the `clone()` syscall so that the new thread will use the same VAS. So the shared memory mechanism for threads is ensured by the Kernel. We don't need to do anything more. The correct answer is, therefore, that the processes offer better confinement due to the use of a Virtual Address Space for each process.

The correct answer is: Processes are a better way of confinement due to individual Virtual Address Spaces.

Question **5**

Correct

Mark 2.00 out of 2.00

A difference between an UNIX socket and a file is:

Select one:

- a. The UNIX socket is used for network communication, and the file is used to store data.
- b. The UNIX socket can also be used in distributed systems, but files can only be used locally.
- c. UNIX sockets file descriptors are not kept in the process's file descriptor table, while files file descriptors are.
- d. The Unix socket is backed up by memory, and the file is usually backed up by storage. ✔

An UNIX socket cannot be used for network communication. UNIX sockets cannot be used in distributed systems. All file descriptors are in the same file descriptor table. The UNIX socket is backed up by memory; there's a buffer in kernel space, whereas the content of a file is on the disk.

The correct answer is: The Unix socket is backed up by memory, and the file is usually backed up by storage.

◄ Curs 11: App Interaction (part 1): Recap

Jump to...

Curs 12: App Interaction (part 2): Recap ►

Started on	Tuesday, 17 January 2023, 5:30 PM
State	Finished
Completed on	Tuesday, 17 January 2023, 5:34 PM
Time taken	3 mins 59 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 2.00 out of 2.00

Can a pipe be used after `execve()` has been called in the child process?

Select one:

- a. Yes, because the `execve()` system call does not close file descriptors by default. ✔
- b. No, because the `execve()` system call closes all file descriptors except STDIN, STDOUT and STDERR.
- c. No, because the `execve()` system call changes the entire Virtual Address Space.
- d. No, because the `execve()` system call closes all file descriptors.

From the `execve()` man page: "By default, file descriptors remain open across an `execve()`. File descriptors that are marked close-on-exec are closed."

The correct answer is: Yes, because the `execve()` system call does not close file descriptors by default.

Question **2**

Correct

Mark 2.00 out of 2.00

What is an advantage of using processes instead of threads when we want to modularize an application?

Select one:

- a. Processes are a better way of confinement due to individual Virtual Address Spaces. ✔
- b. Processes can share memory more easily than threads using the `mmap()` syscall with the `MAP_SHARED` flag.
- c. Processes can run on multiple CPUs, but threads cannot.
- d. There is no advantage of using processes over threads.

Both processes and threads can run on several CPUs (n.b. as long as they are kernel-level threads). Both processes and threads are created using the `clone()` syscall (e.g. `pthread_create()` will eventually call `clone()`). When creating a thread, we specify some flags to the `clone()` syscall so that the new thread will use the same VAS. So the shared memory mechanism for threads is ensured by the Kernel. We don't need to do anything more. The correct answer is, therefore, that the processes offer better confinement due to the use of a Virtual Address Space for each process.

The correct answer is: Processes are a better way of confinement due to individual Virtual Address Spaces.

Question **3**

Correct

Mark 2.00 out of 2.00

Can we use BSD sockets for local communication?

Select one:

- a. Yes. Using BSD sockets bound to local files.
- b. Yes. Using any network interface. ✔
- c. No. Because BSD sockets are network sockets and can only be used for remote communication.
- d. Yes. Only on port 80.

Although BSD sockets are used for network communication, they can also be used locally for communication between processes of the same machine using any local interface (e.g. the loopback one).

The correct answer is: Yes. Using any network interface.

Question **4**

Correct

Mark 2.00 out of 2.00

What is the most efficient method of inter-process communication?

Select one:

- a. Shared Memory. ✔
- b. Message Passing.
- c. Signals.
- d. Reading/Writing to files.

The inter-process communication mechanisms based on shared memory are the most efficient because once we get hold of a portion of shared memory, the effective communication is just working with memory. Unlike shared memory, message passing usually involves `send()` and `recv()` system calls. Signals also involve system calls, and writing and reading from a file involves working with IO, which is slow.

The correct answer is: Shared Memory.

Question **5**

Correct

Mark 2.00 out of 2.00

Can we use semaphores as an inter-thread communication mechanism?

Select one:

a. No. Because we cannot use the `wait()` and `post()` methods in two different threads.

b. Yes. Because we can use the `wait()` and `post()` methods in two different threads. ✔

c. Yes. Because we can use the semaphore for exclusive access to a shared resource.

d. No. Because we cannot send text messages using semaphores.

Inter-thread communication means communication between two or more threads. So while you can use a semaphore to get exclusive access to a shared resource, that's not its primary purpose. Semaphores are used as signalling mechanisms. The `wait()` and `post()` methods are used precisely in this sense. One thread will wait using the `wait()` method, and another will call `post()` to notify the first one. Finally, we can also conclude that communication is not limited to the transfer of text messages.

The correct answer is: Yes. Because we can use the `wait()` and `post()` methods in two different threads.

◄ Curs 11: App Interaction (part 1): Recap

Jump to...

Curs 12: App Interaction (part 2): Recap ►